STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ

ZOCHOVA 9, BRATISLAVA

Golang Ray-Tracer

projekt z praktickej časti odbornej zložky maturitnej skúšky

Meno kandidáta: Matus Bencek **Trieda: 4.D**

Vedúci práce: Mgr. Jakub Krcho **Šk. rok: 2023/2024**

STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ

ZOCHOVA 9, BRATISLAVA

Meno kandidáta: Matus Bencek **Trieda: 4.D**

Vedúci práce: Mgr. Jakub Krcho

Rozsah: 11 stran

Úloha: Navrhnite program, ktorý je schopný renderovať 3D scénu pomocou techniky ray tracing v programovacom jazyku Go. Program bude sledovať svetelné lúče, ktoré prechádzajú cez 3D objekty, pričom bude počítať ich interakcie s povrchmi, ako sú odrazy, tiene a lomy. Hlavným cieľom je dosiahnuť realistické zobrazenie scény na základe presných fyzikálnych výpočtov svetla a jeho správania. Program musí byť optimalizovaný tak, aby dokázal spracovať aj komplexnejšie scény v rozumnom čase.

Čestné prehlásenie

Čestne prehlasujem, že problematiku týkajúcu sa náplne zadaného projektu v rámci praktickej časti odbornej zložky maturitnej skúšky formou obhajoby som spracoval sám za pomocí uvedenej použitej literatúry. Inú, ako uvedenú literatúru, som nepoužil.

V Bratislave dňa: Matus Bencek

Obsah

Úvod 0
2.1 Architektúra Aplikácie 0
2.1.1 Princíp fungovania BVH 1
2.1.2 Surface Area Heuristic (SAH) 1
2.1.3 Implementačné prístupy 1
2.2 Reprezentácia Trojuholníkov a Materiálové Vlastnosti 2
2.2.1 Geometrická Reprezentácia 3
2.2.2 Materiálové Vlastnosti 3
2.3 Podpora Načítavania 3D Geometrie a jej ukladania 5
2.3.1 Načítavanie .OBJ Súborov 5
2.3.2 Ukladanie a Načítavanie BVH Štruktúry 6
2.4 Podpora Post-Processing Shaderov 6
2.4.1 Charakteristika Kage Shader Language 6
2.4.2 Implementované Post-Processing Efekty 6
2.4.3 Architektúra Podpory Shaderov 7
2.5 Systém Benchmarkovania a Výkonnostnej Analýzy 7
2.5.1 Sledované Metriky Výkonu 7
2.5.2 Princípy Implementácie 8
2.5.3 Kľúčové Funkcionality 8
2.5.4 Význam pre Vývoj 8
2.5.5 Mechanizmus Sledovania Histórie 8
2.6.1 Vrstvový Systém 9
2.6.2 Nástroje a Funkcionality 9
Záver
Zdroje 11

1.0 Úvod

V súčasnej dobe počítačová grafika zohráva kľúčovú úlohu v mnohých oblastiach, od herného priemyslu až po vedecké vizualizácie. Jednou z najvýznamnejších technológií v tejto oblasti je Ray-Tracing, ktorý umožňuje vytvárať fotorealistické zobrazenia 3D scén simuláciou fyzikálnych vlastností svetla. Táto maturitná práca sa zameriava na implementáciu vlastného 3D engine-u, ktorý využíva práve túto pokročilú technológiu renderovania.

Hlavným cieľom práce je vytvoriť flexibilný a výkonný 3D engine, ktorý bude schopný nielen základného renderovania 3D scén pomocou Ray-Tracingu, ale poskytne aj možnosť využívať rôzne shadre pre pokročilé vizuálne efekty. Významnou súčasťou projektu je implementácia podpory pre renderovanie volumetrických materiálov prostredníctvom technológie Voxelov , čo ďalej rozširuje možnosti vizualizácie komplexných objektov a efektov.

Pre implementáciu bol zvolený programovací jazyk Golang, ktorý sa vyznačuje niekoľkými kľúčovými výhodami. Prvou je jeho efektívna podpora multiprocesingu prostredníctvom Go rutín, čo je esenciálne pre optimalizáciu výkonu pri ray-tracingu. Druhou výhodou je jeho výkonnosť, ktorá sa približuje tradičným systémovým jazykom ako C a C++. Pre implementáciu shaderových programov bude využitý jazyk Kage, ktorý bol vyvinutý pre Ebiten 2D engine. Kage poskytuje intuitívnu syntax inšpirovanú jazykom Go, čo umožňuje efektívny vývoj shaderov.

Aplikácia poskytne užívateľom možnosť interaktívne upravovať vlastnosti 3D geometrie, vrátane farieb a rôznych aspektov materiálov. Dôraz je kladený na optimalizáciu výkonu, aby bolo možné renderovať scény v realistickom čase. Architektúra systému je navrhnutá s ohľadom na budúcu adaptabilitu, najmä v oblasti podpory rôznych grafických API a efektívneho využívania GPU akcelerácie.

2.1 Architektúra Aplikácie – Implementácia BVH (Bounding Volume Hierarchy)

BVH (Bounding Volume Hierarchy) predstavuje fundamentálnu dátovú štruktúru v moderných ray-traceroch, ktorá zásadným spôsobom optimalizuje proces renderovania 3D scén. Jej hlavným princípom je hierarchické delenie priestoru scény na menšie podpriestory, čím sa výrazne redukuje počet potrebných výpočtov pri sledovaní lúčov.

2.1.1 Princíp fungovania BVH

Pri ray-tracingu je kľúčovou operáciou hľadanie priesečníkov medzi lúčom vyslaným z kamery a objektmi v scéne. Bez BVH štruktúry by bolo potrebné testovať každý lúč s každým objektom v scéne, čo by viedlo k časovej zložitosti O(n) pre každý lúč, kde n je počet objektov v scéne. BVH rieši tento problém vytvorením hierarchickej štruktúry obaľujúcich objemov (najčastejšie osovo zarovnaných boxov - AABB), ktorá umožňuje rýchlo eliminovať veľké časti scény, ktoré lúč nemôže zasiahnuť.

Keď lúč prechádza scénou, najprv sa testuje prienik s head Node BVH. Ak lúč nezasiahne obsah Nody môžeme okamžite preskočiť všetky objekty v tomto podstrome. Ak prienik existuje, algoritmus rekurzívne pokračuje do potomkov uzla, až kým nedosiahne listové uzly obsahujúce konkrétne objekty scény.

2.1.2 Surface Area Heuristic (SAH)

Pre optimálny výkon BVH je kľúčové, ako sa scéna rozdelí na podpriestory. Tu prichádza do hry Surface Area Heuristic (SAH). Táto heuristika optimalizuje rozdelenie objektov medzi children každéj Node na základe plochy ich objemov. Cieľom je minimalizovať čas potrebný na prechádzanie stromom a testovanie prienikov.

SAH pracuje na princípe, že pravdepodobnosť, že lúč zasiahne dany Box, je úmerná jeho povrchu. Pri delení Nody sa teda snažíme minimalizovať funkciu:

C = Ct + (SA(L)/SA(P)) * NL * Ci + (SA(R)/SA(P)) * NR * Ci

kde:

- Ct je cena prechodu cez Node
- Ci je cena testovania prieniku s objektom
- SA(X) je plocha povrchu objemu
- NL a NR sú počty objektov v ľavom a pravom potomkovi
- L, R, P označujú ľavého potomka, pravého potomka a parrent Node

2.1.3 Implementačné prístupy

V projekte som experimentoval s dvoma prístupmi k implementácii BVH:

a) Klasická reprezentácia Tradičný prístup využíva Node štruktúru, kde každý uzol obsahuje odkazy na svojich potomkov (pointer):

```go
type BVHNode struct {
    boundingBox AABB
    leftChild   *BVHNode
    rightChild  *BVHNode
    objects     []Object
}
```

Táto implementácia je intuitívna a flexibilná, ale môže trpieť nižšou cache lokalitou kvôli rozptýleným prístupom do pamäte.

b) Lineárna reprezentácia v poli je alternatívny prístup a využíva reprezentáciu binárneho stromu v jednorozmernom poli, kde pre Node na indexe n:

- ľavý potomok sa nachádza na indexe 2*n
- pravý potomok na indexe 2*n + 1

```go
type BVHArray struct {
    nodes []BVHNode
}
```

Táto implementácia prináša niekoľko významných výhod:

1. Lepšia cache lokalita - Nody sú uložené súvislo v pamäti
2. Efektívnejšie využitie pamäte - nie je potrebné ukladať odkazy na potomkov
3. Rýchlejšia serializácia a deserializácia

Nevýhodou môže byť menšia flexibilita pri modifikácii stromu a potenciálne väčšia spotreba pamäte pri nevyvážených stromoch.

V aktuálnej implementácii som sa priklonil k lineárnej reprezentácii, pretože jej výhody v oblasti výkonu a cache locality prevažujú nad nevýhodami, hlavne v kontexte ray-tracingu, kde je štruktúra BVH väčšinou statická počas renderovania.

2.2 Reprezentácia Trojuholníkov a Materiálové Vlastnosti

Základným stavebným prvkom 3D scény v implementovanom ray-traceri je trojuholník, ktorý je reprezentovaný štruktúrou TriangleSimple. Táto štruktúra kombinuje

geometrické vlastnosti trojuholníka s jeho materiálovými charakteristikami, čo umožňuje realistické zobrazenie rôznych povrchov a materiálov.

2.2.1 Geometrická Reprezentácia

```
type TriangleSimple struct {
    v1, v2, v3 Vector    // Vrcholy trojuholníka
    Normal     Vector    // Normálový vektor
    // ... materiálové vlastnosti
}
```

Geometria trojuholníka je definovaná tromi 3D vektormi (v1, v2, v3), ktoré predstavujú jeho vrcholy v priestore. Pre optimalizáciu výkonu je súčasťou štruktúry aj predpočítaný normálový vektor (Normal). Tento prístup významne urýchľuje proces renderovania, keďže normál Vector je kľúčová pri výpočtoch osvetlenia a nie je potrebné ju opakovane počítať pri každom prieniku lúča s trojuholníkom.

2.2.2 Materiálové Vlastnosti

Materiálové vlastnosti trojuholníka sú reprezentované niekoľkými kľúčovými parametrami, ktoré určujú jeho vizuálne charakteristiky:

a) **Farba** (color ColorFloat32)

```
type ColorFloat32 struct {
    R, G, B, A float32
}
```

Farba povrchu je reprezentovaná pomocou vlastnej štruktúry ColorFloat32, ktorá využíva pre každý farebný kanál (červený, zelený, modrý) a alfa kanál hodnoty typu float32. Toto riešenie prináša niekoľko kľúčových výhod oproti tradičnej RGBA reprezentácii (uint8):

1. Vysoký Dynamický Rozsah (HDR):

- Na rozdiel od štandardnej RGBA reprezentácie, kde je každý kanál limitovaný rozsahom 0-255 (uint8), float32 umožňuje reprezentovať hodnoty výrazne presahujúce hodnotu 1.0
- Toto je esenciálne pre realistické zobrazenie emisívnych materiálov, ktoré môžu vyžarovať svetlo s intenzitou mnohonásobne vyššou než 1.0
- Umožňuje presnejšie zachytenie a reprezentáciu svetelných efektov v scéne

2. Emisívne Materiály:
    - ColorFloat32 umožňuje definovať materiály, ktoré aktívne emitujú svetlo do scény
    - Hodnoty vyššie ako 255 reprezentujú materiály, ktoré pridávajú energiu do scény
    - Toto je kľúčové pre implementáciu svetelných zdrojov priamo ako súčasti geometrie scény
3. Presnosť Výpočtov:
    - Float32 poskytuje vyššiu presnosť pri výpočtoch s farbami
    - Eliminuje sa problém kvantizácie, ktorý je typický pre uint8 reprezentáciu
    - Umožňuje jemnejšie prechody a gradienty v renderovanom obraze
4. Fyzikálna Korektnosť:
    - Reprezentácia pomocou float32 lepšie zodpovedá fyzikálnej realite, kde intenzita svetla nie je zhora obmedzená
    - Umožňuje presnejšiu simuláciu svetelných interakcií v scéne
    - Podporuje fyzikálne korektné miešanie farieb a svetelných príspevkov

Táto implementácia je kľúčová pre dosiahnutie fotorealistického renderovania, keďže umožňuje pracovať s realistickými svetelnými podmienkami a materiálmi, ktoré by nebolo možné reprezentovať v štandardnom 8-bitovom farebnom priestore. Zároveň poskytuje základ pre implementáciu pokročilých renderovacích techník ako HDR rendering a tone mapping.

b) D**irect-to-Scatter Ratio** (directToScatter float32)

Tento parameter, definovaný v rozsahu [0, 1], určuje pomer medzi priamym odrazom svetla a difúznym rozptylom:

- Hodnota blízka 0: Väčšina svetla je rozptýlená náhodným smerom (matný povrch)
- Hodnota blízka 1: Prevláda priamy odraz svetla (lesklý povrch) Tento parameter je kľúčový pre realistické zobrazenie rôznych typov materiálov, od matných až po vysoko lesklé povrchy.

c) **Reflection Coefficient** (reflection float32) Koeficient odrazu, definovaný v rozsahu [0, 1], určuje, ako silno povrch odráža okolité prostredie:

- 0: Žiadne odrazy okolitého prostredia
- 1: Dokonalé zrkadlové odrazy Tento parameter ovplyvňuje pomer medzi vlastnou farbou objektu a farbou odrazenou z okolia, čo umožňuje simulovať materiály od úplne matných až po zrkadlové povrchy.

d) **Specular Intensity** (specular float32) Parameter v rozsahu [0, 1] určuje intenzitu spekulárneho odrazu:

- 0: Žiadny spekulárny odraz
- 1: Maximálny spekulárny odraz Tento parameter je kľúčový pre vytvorenie realistických highlight-ov a lesklých efektov na povrchu materiálu.

2.3 Podpora Načítavania 3D Geometrie a jej ukladania

2.3.1 Načítavanie .OBJ Súborov

Implementovaný ray-tracer poskytuje robustnú podporu pre načítavanie 3D geometrie prostredníctvom štandardného .obj formátu, čo výrazne zvyšuje flexibilitu a použiteľnosť aplikácie.

Kľúčové vlastnosti implementácie:

1. Podpora Geometrie:
    - Načítavanie priestorových vrcholov (vertices)
    - Extrakcia normálových vektorov
    - Podpora textúrovacích koordinát
    - Konverzia polygónov na trojuholníkovú sieť
2. Podpora Materiálov:
    - Parsing .mtl súborov
    - Načítavanie základných materiálových vlastností:
        - Difúzna farba
        - Odrazivosť
        - Spekulárne vlastnosti
        - Priehľadnosť
3. Optimalizačné Techniky:
    - Predpočítavanie normálových vektorov
    - Efektívna konverzia na interný formát TriangleSimple
    - Podpora pre zložitejšie geometrické útvary

Proces načítavania .obj súborov zahŕňa niekoľko kľúčových krokov:

- Parsovanie priestorových súradníc vertices
- Extrahovanie normálových vektorov
- Identifikácia a konverzia polygónov na trojuholníky

- Priradenie materiálových vlastností jednotlivým geometrickým prvkom

2.3.2 Ukladanie a Načítavanie BVH Štruktúry

Pre ďalšiu optimalizáciu výkonu je implementovaná funkcionalita serializácie a deserializácie BVH štruktúry.

Kľúčové výhody:

1. Výkonová Optimalizácia:
    - Eliminácia potreby opätovného výpočtu BVH pre statické scény
    - Významné zníženie inicializačného času aplikácie
2. Flexibilita Ukladania:
    - Podpora serializácie do binárneho formátu
    - Možnosť kompresie uložených dát
3. Výhody Implementácie:
    - Rýchle načítanie komplexných scén
    - Úspora výpočtového času
    - Možnosť predspracovania scén mimo hlavnej aplikácie

Proces serializácie BVH štruktúry umožňuje:

- Uloženie kompletnej hierarchickej štruktúry priestorového rozdelenia scény
- Zachovanie optimalizačnej štruktúry pre následné použitie
- Rýchle obnovenie už predspracovanej scény bez nutnosti opätovného výpočtu

Spomenuté riešenie poskytuje flexibilnú a výkonnú metódu načítavania 3D geometrie a optimalizácie štruktúry scény, pričom umožňuje jednoduchú integráciu s existujúcimi 3D modelovacími nástrojmi.

2.4 Podpora Post-Processing Shaderov

Post-processing shadre predstavujú kľúčový prvok pre vizuálne vylepšenie výstupného obrazu v ray-traceri. Implementácia využíva Kage shader language, ktorý bol vyvinutý súbežne s Ebiten 2D enginom a vyznačuje sa syntaxou inšpirovanou programovacím jazykom Go.

2.4.1 Charakteristika Kage Shader Language

Kage prináša niekoľko podstatných výhod:

- Syntax podobná jazyku Go, čo uľahčuje písanie shaderov
- Silná typová kontrola
- Priama integrácia s grafickými knižnicami
- Jednoduchá čitateľnosť a údržba kódu

2.4.2 Implementované Post-Processing Efekty

Aktuálne implementované post-processing shadre zahŕňajú:

1. Sharpness
    - Umožňuje zvýrazniť detaily obrazu
    - Nastaviteľná intenzita efektu
    - Vhodné pre zvýraznenie jemných štruktúr v renderovanej scéne
2. Tint
    - Pridáva globálny farebný nádych na celý obraz
    - Umožňuje jemné farebné korekcie
    - Flexibilné nastavenie farby a intenzity
3. Dithering
    - Technika na redukciu farebnej hĺbky obrazu
    - Pridáva štrukturovaný šum pre simuláciu vyššej farebnej hĺbky
    - Vhodné pre štylizované vizuálne efekty
4. Contrast
    - Umožňuje dynamickú úpravu kontrastu obrazu
    - Nastaviteľný rozsah zvýraznenia rozdielov medzi svetlými a tmavými oblasťami
    - Pomáha zvýrazniť detaily v renderovanej scéne
5. Bloom
    - Pridáva mäkké svetelné aureoly okolo jasných oblastí
    - Nastaviteľná intenzita a rozsah efektu

2.4.3 Architektúra Podpory Shaderov

Implementácia podporuje flexibilnú architektúru pre:

- Konfiguráciu parametrov jednotlivých efektov
- Reťazenie viacerých post-processing efektov

Navrhnutá architektúra shaderov umožňuje jednoduchú rozšíriteľnosť a pridávanie nových vizuálnych efektov s minimálnymi zmenami v existujúcom kóde.

2.5 Systém Benchmarkovania a Výkonnostnej Analýzy

Implementovaný benchmarkový systém predstavuje kľúčový nástroj pre sledovanie výkonu ray-tracera počas vývoja. Jeho hlavným cieľom je poskytnúť komplexný pohľad na výkonnostné charakteristiky aplikácie.

2.5.1 Sledované Metriky Výkonu

Systém zaznamenáva niekoľko kľúčových výkonnostných ukazovateľov:

1. Priemerný počet snímok za sekundu (FPS)
    - Základný indikátor celkového výkonu aplikácie
    - Poskytuje celkový pohľad na plynulosť renderovania
2. Maximálny FPS
    - Identifikuje špičkový výkon systému

    - Užitočný pre hodnotenie optimálnych scenárov
3. Minimálny FPS
    - Odhaľuje kritické body z hľadiska výkonu
    - Kľúčový pre identifikáciu potenciálnych výkonnostných problémov
4. Priemerný 15% Low FPS
    - Eliminuje náhodné výkyvy
    - Poskytuje stabilnejší pohľad na reálny výkon
    - Lepšie reprezentuje užívateľskú skúsenosť

2.5.2 Princípy Implementácie

Benchmarkový systém je navrhnutý s dôrazom na:

- Minimálny dopad na celkový výkon aplikácie
- Presné meranie výkonnostných charakteristík
- Uchovávanie histórie výkonu
- Jednoduchú integráciu do vývojového procesu

2.5.3 Kľúčové Funkcionality

Systém umožňuje:

- Automatické zaznamenávanie výkonnostných metrík
- Porovnávanie výkonu medzi rôznymi verziami kódu
- Identifikáciu regresií vo výkone
- Analýzu vplyvu zmien v implementácii

2.5.4 Význam pre Vývoj

Hlavné prínosy benchmarkového systému:

- Priebežné monitorovanie výkonu počas vývoja
- Rýchla identifikácia výkonnostných problémov
- Možnosť presného porovnávania rôznych implementačných prístupov
- Podpora rozhodovania o optimalizáciách

2.5.5 Mechanizmus Sledovania Histórie

Systém uchováva históriu výkonu, čo umožňuje:

- Sledovanie trendov výkonu v čase
- Analýzu dopadu jednotlivých commitov na výkon
- Možnosť návratu k výkonnejším verziám kódu v prípade zistenia regresie

Princíp je postavený na zaznamenávaní výkonnostných metrík pre každú významnú zmenu v kóde, čo umožňuje presné sledovanie vývoja výkonu aplikácie.

Navrhnutý benchmarkový systém tak predstavuje robustný nástroj pre kontinuálne monitorovanie a optimalizáciu výkonu ray-tracera počas celého vývojového procesu.

2.6.1 Vrstvový Systém

Implementovaný 2D grafický systém je navrhnutý ako viacvrstvová kompzičná platforma, ktorá umožňuje flexibilné upravovanie renderovaného obrazu. Kľúčové charakteristiky:

- Podpora vrstiev
- Možnosť nezávislej manipulácie s jednotlivými vrstvami
- Rôzne módy miešania farieb
- Plná podpora alfa kanála pre priehľadnosť

2.6.2 Nástroje a Funkcionality

Systém poskytuje komplexnú sadu nástrojov pre 2D úpravy:

1. Kresliacie Nástroje
    - Rôzne typy štetcov
    - Nastaviteľná veľkosť
    - Dynamická zmena opacity
2. Módy Miešania Farieb
3. Detekcia Hrán
    - Implementované algoritmy detekcie hrán
    - Sobellov operátor

Navrhnutý systém poskytuje robustnú platformu pre post-processing a kreatívne úpravy renderovaného obrazu s dôrazom na flexibilitu.

3.0 Záver

Predložená maturitná práca predstavuje komplexný návrh a implementáciu 3D ray-tracingového engine-u, ktorý prekračuje tradičné hranice počítačovej grafiky. Projekt nie je iba technickým cvičením, ale ukazuje potenciál pre vytváranie sofistikovaných vizualizačných nástrojov s dôrazom na výkon, flexibilitu a užívateľskú rozšíriteľnosť.

Kľúčové prínosy práce:

1. Technologická Inovácia
    - Implementácia pokročilých ray-tracingových techník
    - Podpora komplexných renderovacích algoritmov
    - Flexibilný systém pre volumetrické a 3D zobrazovanie
2. Architektonické a Výkonnostné Riešenia
    - Optimalizačné štruktúry ako BVH
    - Efektívne využitie multiprocesingu
    - Podpora štandardných 3D formátov
    - Robustný benchmarkový systém pre kontinuálne meranie výkonu

3. Rozšírené Grafické Možnosti
    - Pokročilý post-processing
    - Podpora shaderových efektov
    - 2D vrstvový systém pre následné úpravy
    - Flexibilné nástroje pre manuálne a procedurálne úpravy obrazu

Projekt poskytuje nielen technické riešenie, ale aj platformu pre ďalší výskum a vývoj v oblasti počítačovej grafiky. Ukazuje, že moderné programovacie techniky a hlboké pochopenie grafických algoritmov môžu vyústiť do výkonného a adaptabilného grafického systému.

Perspektívy ďalšieho vývoja:

- Integrácia pokročilých renderovacích techník
- Podpora real-time ray-tracingu
- Implementácia fyzikálne presnejších light transportných modelov
- Podpora komplexnejších animačných a dynamických scén

Implementovaný engine nie je len akademickým projektom, ale solidným základom pre budúci vývoj sofistikovaných grafických nástrojov. Demonstruje schopnosť navrhnúť komplexný systém, ktorý kombinuje výkonnosť, flexibilitu a inovatívny prístup k počítačovej grafike.

4.0 Zdroje

- Ray Tracing in One Weekend

  https://raytracing.github.io/books/RayTracingInOneWeekend.html#overview

- Ray Tracing: The Next Week

  https://raytracing.github.io/books/RayTracingTheNextWeek.html

- Ray Tracing: The Rest of Your Life

  https://raytracing.github.io/books/RayTracingTheRestOfYourLife.html#cleaningupppdfmanagement/diffuseversusspecular

- Why you should avoid Linked List

  https://www.youtube.com/watch?v=YQs6IC-vgmo

- Why is recursion bad?

  https://www.youtube.com/watch?v=mMEmNX6aW_k

- How Big Budget AAA Games Render Bloom

  https://www.youtube.com/watch?v=ml-5OGZC7vE

- Andrew Kelley Practical Data Oriented Design (DoD)

  https://www.youtube.com/watch?v=IroPQ150F6c

- I redesigned my game

  https://www.youtube.com/watch?v=PcMua73C_94